
Micro-Architecture, Performance, and Resource Utilization Review

1. Potential Deadlock: wait_to_verify FIFO is undersized for large packets

Severity: Critical

Files: wait_to_verify.c:28, decrypt_dataflow.c:37

```
GLOBAL_STREAM_FIFO(axis128_t, verify_fifo, 128) // 2 KB
```

The decrypt datapath forks the ciphertext stream to two consumers simultaneously:

```
strip_auth_tag_axis_out_ready = prep_auth_data_decrypt_axis_in_ready  
                                & chacha20_decrypt_axis_in_ready;
```

ChaCha20 is significantly faster than Poly1305 (10.24 Gbps vs 2.048 Gbps). Between the time ChaCha20 starts producing plaintext and the time Poly1305 finishes computing the auth tag, the FIFO accumulates plaintext that cannot drain (the wait_to_verify FSM holds verify_fifo_out_ready=0 until the verify bit arrives). The maximum FIFO occupancy is roughly:

$$\text{FIFO_fill_words} \approx (\text{Poly1305_latency_cycles} - \text{ChaCha20_pipeline_latency}) / 4$$
$$\approx (5 \times N_blocks + \text{overhead} - 31) / 4$$

For a 1500-byte Ethernet frame (~94 16-byte blocks): fill $\approx$ 100–120 words — close to the 128-word limit. For a 4096-byte packet: fill $\approx$ 290 words — overflow.

When the FIFO fills, wait_to_verify_axis_in_ready=0 stalls ChaCha20 output. That stall propagates back through the ciphertext fork:

strip_auth_tag_axis_out_ready goes to 0, which stalls the prep_auth_data input as well. Poly1305 stops receiving data and never produces the auth tag. The FIFO never drains. This is a deadlock.

For WireGuard protocol compliance the FIFO must accommodate the maximum allowed plaintext size (65535 bytes = 4096×16 -byte words). At 128-bit word width that requires approximately 4096 entries, or $14\times$ the current depth. This can be implemented with 8 BRAM36 blocks on the Artix-7, which is well within the device budget.

Fix:

```
GLOBAL_STREAM_FIFO(axis128_t, verify_fifo, 4096) // 64 KB
```

Alternatively, the design could guarantee safety by limiting packet size via the CSR (with the current design, documented as a hard 2 KB constraint).

2. Poly1305 arithmetic: 64-bit limbs consume $6\times$ more DSPs than necessary

Severity: High — resource

File: poly1305.h:28, poly1305_mac.c:26

```
typedef struct u320_t {  
    uint64_t limbs[5]; // five 64-bit limbs  
} u320_t;
```

The schoolbook uint320_mul computes up to 25 partial 64×64 products. On the Artix-7, a 64×64 multiply requires multiple DSP48E1 cascades (each DSP48E1 does 18×25 signed bits). This is why Table I shows 150 DSP48E1 per core.

The standard high-efficiency representation for Poly1305 arithmetic is five 26-bit limbs (radix- 2^{26}). With this representation:

- Each partial product is $26 \times 26 = 676$ bits — fits in a single DSP48E1 (25×18 signed)
- $5 \times 5 = 25$ partial products → 25 DSP48E1 per Poly1305 instance
- The clamped r has only 124 effective bits, further reducing carry propagation

Estimated savings: 150 → ~30 DSP48E1 per instance (5× reduction), freeing ~120 DSPs per core. On the Artix-7 200T with 740 total DSPs, the current design (150 per standalone core) already uses 20% of all DSPs for Poly1305 alone.

The trade-off is more reduction logic in LUTs (handling the 2^{26} carries), but the net resource profile is far more FPGA-appropriate. This is also the representation used by djb's reference software and most efficient FPGA Poly1305 implementations.

3. Poly1305 MCP: 4→3 cycles is achievable with restructured arithmetic

Severity: Medium — performance

The paper identifies 3-cycle MCP as future work. The bottleneck in `poly1305_mac_loop_body` is the combined multiply-and-reduce path. With the 26-bit limb representation from item 2:

- 25 partial products each fit in one DSP (registered output available in 1 cycle)
- The partial product accumulation tree has depth $\approx \log_2(25) \approx 5$ LUT levels
- Reduction mod 2^{130-5} exploits the near-power-of-two structure (one multiply-by-5 via shift-and-add)

This restructuring would likely allow a 3-cycle MCP constraint, reducing feedback latency from 5 to 4 cycles:

Throughput = $10.24 \text{ Gbps} / 4 = 2.56 \text{ Gbps}$ (+25% over current 2.048 Gbps)

4. ChaCha20 round-robin arbiter wastes slots during unidirectional traffic

Severity: Medium — performance

File: chacha20_pipeline_shared.c:53-64

```
static uint1_t is_encrypt;  
  
// ... select stream based on is_encrypt ...  
  
is_encrypt = ~is_encrypt; // toggles every cycle unconditionally
```

The arbiter toggles every clock cycle regardless of whether the selected stream has valid data. During unidirectional traffic (e.g., bulk download: all traffic is one direction), half the pipeline slots are offered to the idle stream and wasted. The shared pipeline runs at

50% utilization instead of 100%.

Fix — demand-driven with round-robin fairness:

```
static uint1_t rr_state;  
  
uint1_t selected = rr_state;  
  
// Fall back to other stream if selected has no data  
if(!selected_stream.valid && other_stream.valid)  
    selected = ~selected;  
  
// Only advance round-robin on actual transfer  
if(pipeline_accepted_a_transaction)  
    rr_state = ~rr_state;
```

This maintains fairness under balanced bidirectional traffic while achieving near-100% utilization under unidirectional load, effectively doubling throughput in that scenario.

5. Per-packet Poly1305 key generation adds a fixed pipeline-depth latency

Severity: Medium — performance (short packets)

File: chacha20.h:305-318

At the start of every packet the ChaCha20 FSM sends one all-zero 64-byte block (counter=0) into the pipeline to generate the Poly1305 key:

```
if(input_side_state == POLY_KEY){  
    // send zero block with block_count=0 into pipeline  
  
    ARRAY_SET(o.to_pipeline.data.axis_in.tdata, 0, CHACHA20_BLOCK_SIZE)  
  
    o.to_pipeline.valid = 1;  
  
    if(to_pipeline_ready){  
        block_count += 1;  
        input_side_state = PLAINTEXT;  
    }  
}
```

The Poly1305 key emerges from the pipeline only after a full pipeline latency (~31 cycles). Until that latency has elapsed, no plaintext can flow through ChaCha20 and Poly1305 cannot start. For a 64-byte packet (one ChaCha20 block), the key gen adds 100% overhead. For a 1500-byte frame (~24 blocks), it adds ~4%.

Pre-computation optimization: since the nonce is written via CSR before the packet arrives, the key generation block can be dispatched into the pipeline immediately when the nonce CSR is written, with the resulting 32-byte Poly1305 key cached in registers. When the next packet arrives, Poly1305 starts immediately without waiting for the key pipeline pass. This would recover 31+ cycles per packet and is a pure win for short-packet performance.

6. poly1305_verify_decrypt wastes one cycle in the COMPARE_TAGS state

Severity: Low — performance

File: poly1305_verify_decrypt.c:74-81

```
else if (state == COMPARE_TAGS)
{
    tags_match_reg = (auth_tag_reg == calc_tag_reg);

    state = OUTPUT_COMPARE_RESULT;
}
```

This state does exactly one thing — a 128-bit equality comparison — and produces no external output. The comparison is combinational and can be done in the same clock cycle that accepts calc_tag_reg in TAKE_CALC_TAG, eliminating a wasted cycle at the end of every decryption:

```
// In TAKE_CALC_TAG, on accepting transfer:

calc_tag_reg = poly1305_verify_calc_tag.data;

tags_match_reg = (auth_tag_reg == poly1305_verify_calc_tag.data);

state = OUTPUT_COMPARE_RESULT; // skip COMPARE_TAGS entirely
```

The FSM drops from 4 states to 3.

7. ChaCha20 pipeline wrapper depth (64) likely over-provisioned

Severity: Low — resource

File: chacha20.c:34, chacha20_pipeline_shared.c:29

```
GLOBAL_VALID_READY_PIPELINE_INST(..., 64)
```

The 64 argument is the depth of the valid/ready handshake buffer that wraps the pipeline. If the actual auto-pipelined depth is 31 stages (as the paper claims), then a buffer depth of 32 would be sufficient to prevent stalls. Halving from 64 to 32 would halve the BRAM or flip-flop usage for this buffer — noteworthy because the ChaCha20 pipeline operates on 520-bit words (512-bit data + 8-bit control for the shared version), making each FIFO entry large.

At 520-bit × 64-entries = 33,280 bits: this fits in one BRAM36 with room to spare. Reducing to 32 entries saves that BRAM36, recovering 1 of the 10 BRAM36 blocks visible in Table I.

8. prep_auth_data CIPHERTEXT counter is uint16_t with no assertion

Severity: Low — correctness

File: prep_auth_data.h:35

```
static uint16_t counter;
```

The counter accumulates ciphertext byte count for the LENGTHS block. uint16_t caps at 65535 bytes, which matches WireGuard's maximum per-packet payload. However the constraint is implicit and silent — an oversized packet would produce a silently wrapped counter and a wrong auth tag without any error signal. A simple compile-time statement or an overflow-detect wire would make this constraint explicit and observable.

9. strip_auth_tag lookahead adds one cycle of fixed latency

Severity: Low — latency

File: strip_auth_tag.c:27-65

The axis128_early_tlast function always buffers one word regardless of whether the lookahead is actually needed. The first word of every packet spends one cycle sitting in buffer_reg before it can flow to the output. For a 1500-byte packet this adds $1/94 \approx 1\%$ latency overhead; for small packets it is more significant. A two-entry shift register that also peeks at the current input word would provide the same early_tlast signal with the same area cost but without the unconditional one-cycle stall for single-word packets.

Summary Table

#	Item	Category	Impact
1	FIFO undersized → deadlock for packets > ~2 KB	Correctness/Arch	Critical
2	64-bit Poly1305 limbs: 6× too many DSPs	Resource	High
3	Poly1305 MCP 4→3 cycles with 26-bit limbs	Performance	Medium
4	Blind round-robin wastes slots in unidirectional traffic	Performance	Medium
5	Per-packet Poly1305 key gen: pre-compute on nonce CSR write	Performance	Medium
6	COMPARE_TAGS FSM state wastes 1 cycle per packet	Performance	Low
7	ChaCha20 pipeline wrapper depth 64 vs 32 sufficient	Resource	Low
8	uint16_t counter: no overflow detection	Robustness	Low
9	strip_auth_tag lookahead: 1-cycle fixed latency	Latency	Low

The two highest-priority items are not about squeezing performance — they are fundamental: the FIFO deadlock (item 1) renders the design unreliable for any message above approximately 2 KB, and the DSP overuse (item 2) consumes 40% of the Artix-7 200T's DSP budget for Poly1305 alone, which directly limits how many crypto cores could ever be instantiated.